

КЕЙС ХАКАТОНА · ИИ НА ЖЕЛЕЗНОЙ ДОРОГЕ

«Автодиспетчер»

Автоматизация управления движением поездов:
автодиспетчеризация и автоведение

Бесконфликтное планирование движения + рекомендации по энергоэффективному ведению поезда



Диспетчер держит в голове весь участок

Расписание, взаимное расположение поездов, состояние сигналов и стрелок, приоритеты поездов, резервы пропускной способности. При росте интенсивности движения объём решений, которые нужно принимать вручную и быстро, растёт с каждым поездом на участке.



Ручные приоритеты

Диспетчер вручную решает, кому уступить на скрещении и куда направить обгон — под давлением времени.



Ведение «на опыте»

Экономичность и точность прибытия сильно зависят от опыта конкретного машиниста, а не от расчёта.



Цепочка задержек

Одно отклонение от графика вручную пересчитывается «на глаз» и распространяется на весь участок.

Диспетчеру сложно вручную находить бесконфликтный график для всех поездов участка, а ведение поезда редко бывает энергетически и по времени оптимальным.

Ручное распределение приоритетов на скрещениях и обгонах и неоптимальное ведение поезда ведут к потерям времени, перерасходу энергии и снижению пропускной способности участка.

⚠ Прототип — консультативная система поддержки решений; не заменяет СЦБ и системы безопасности движения.



Суть решения

Автодиспетчеризация строит бесконфликтный график, а модуль автоведения подсказывает оптимальную скорость на каждом участке.

Что должен уметь прототип «Автодиспетчер»

01

Данные в реальном времени

Агрегирует положение поездов, состояние сигналов/стрелок и расписание по участку.

02

Автодиспетчеризация

Строит и пересчитывает бесконфликтный график: очерёдность на скрещениях и обгонах, приоритеты.

03

Автоведение (advisory)

Рассчитывает рекомендуемый профиль скорости — энергоэффективность и точность прибытия.

04

Эффективность движения

Эффективность управления поездами на участке

05

Авто-перепланирование

При сбоях — опоздание, неисправность сигнала, закрытие перегона — предлагает альтернативы.

Автодиспетчеризация + автоведение



Потоковые данные

Положение поездов, сигналы/стрелки, расписание — по WebSocket/SSE, от 1 Гц и выше (мок допустим).



Автодиспетчеризация

Constraint-based scheduling, эвристики или RL — бесконфликтный график с учётом приоритетов.



Автоведение (ATO advisory)

Рекомендуемый профиль скорости по участку — энергоэффективность и точность прибытия.



Пересчёт при сбое

Новый план — с оценкой времени и влияния на индекс качества движения для каждого варианта.

Единый интегральный показатель состояния участка



Топ-факторы, влияющие на оценку

Соблюдение расписания



Использование пропускной способности



Энергоэффективность ведения



Число конфликтов на скрещениях/обгонах

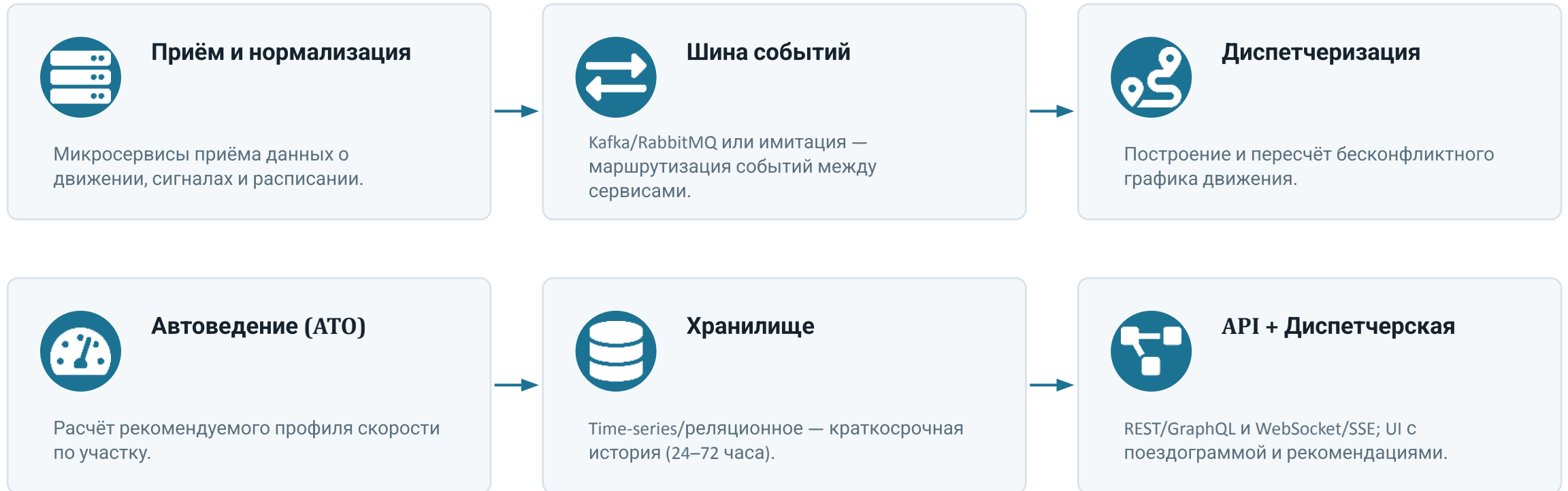


Точность прибытия/остановки



Формула и веса параметров конфигурируются без перекомпиляции; категории «Норма / Внимание / Критично» задаются порогами.

От потока движения до рекомендаций диспетчеру



Скорость, надёжность, безопасность

< 500 мс

Видимое обновление UI от поступления события

≤ 5 сек.

Пересчёт бесконфликтного графика при сбое

x5–x10

Одновременных нештатных ситуаций без просадки UI



Безопасность

Базовая аутентификация, ограничение доступа к настройкам и параметрам оптимизации.



Документация

OpenAPI/Swagger и краткая архитектурная диаграмма.

Что команда показывает на защите

- ✓ Запущенное веб-приложение с симулятором движения поездов по участку
- ✓ Дашборд с поездограммой, картой участка и индексом качества движения
- ✓ Демонстрация автоматического перепланирования при нештатной ситуации
- ✓ Рекомендованный профиль скорости (автоведение) для выбранного поезда
- ✓ «Перемотка» последних 5–15 минут и выгрузка мини-отчёта (PDF/CSV)

Как оценивается прототип



Дизайн интерфейса (UI/UX) — 30%

Читабельность, приоритизация сигналов, информативные визуализации, доступность.

Автодиспетчеризация, автоведение, realtime — 35%

Бесконфликтность плана, качество рекомендаций автоведения, скорость перепланирования, устойчивость.

Архитектура backend — 25%

Модульность, масштабируемость, хранение истории, прозрачность логики.

Демо и инж. культура — 10%

Чистота репозитория, инструкции запуска, логи и метрики, тестовый сценарий.

Как команда сдаёт результат

Рекомендуемый стек

- Frontend: React/Vue/Svelte, ECharts/Recharts, SVG-поездограмма
- Realtime: WebSocket/SSE, Socket.IO
- ИИ/оптимизация: OR-Tools, эвристики, при желании RL
- Backend: Node.js/NestJS, Python/FastAPI, Go/Fiber
- Хранилище: TimescaleDB/PostgreSQL, InfluxDB, SQLite
- Инфра: Docker Compose, Swagger/OpenAPI

Формат сдачи



Git-репозиторий с исходным кодом и инструкциями запуска



Презентация (10–12 слайдов) с архитектурой и формулой индекса



Запись демо или live-показ на защите

Не использовать реальные коммерческие данные без согласования · task-данные по умолчанию · секреты — только через переменные окружения

⚠ Демонстрационное решение — не заменяет сертифицированные системы обеспечения безопасности движения (СЦБ, АЛСН/АЛС-ЕН)

Контакты ответственного со стороны компании — заполняются при публикации кейса (ФИО, должность, e-mail, телефон)